Dynamic content and forms

2C. Dynamic Content & Forms

Questions: 3 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Microsoft, Atlassian, Razorpay, Swiggy

These three questions are about making dynamic, JavaScript-driven UIs accessible. Static HTML accessibility is the baseline — the real challenge is when content changes, options appear and disappear, errors pop up, and the user needs to know about all of it without seeing the screen.

Q 2.8 What are `aria-live` regions and when would you use them?

Difficulty: Mid **Asked at:** Microsoft, Google, Amazon, Flipkart

Live regions are elements whose content changes dynamically, and you want screen readers to announce those changes automatically — without the user needing to navigate to the element.

Consider a chat application. New messages appear at the bottom of the conversation. A sighted user notices them visually. A screen reader user who's focused on the input field gets no indication that a message arrived unless you tell the screen reader to announce it.

HTML Copy

```
  <div class="message">Bob: Let me check my calendar.</div>
  <!— When JS adds this new message, screen reader announces it automatic
</div>
```

The two values that matter:

`aria-live="polite"` — The screen reader finishes whatever it's currently saying, then announces the change. This is the default for most updates. Use it for: new chat messages, search result counts updating, toast notifications, status changes ("3 items in cart"), content loading.

`aria-live="assertive"` — The screen reader interrupts whatever it's saying and announces the change immediately. Reserve this for urgent information the user needs to know right now. Use it for: form validation errors, session timeout warnings, critical alerts ("Payment failed"), error banners.

HTML Copy

```
<!— Polite: Search result count updating — not urgent —>
<div aria-live="polite" role="status">
  Showing 42 results for "wireless headphones"
</div>

<!— Assertive: Error that needs immediate attention —>
<div aria-live="assertive" role="alert">
  Your session expires in 2 minutes. Save your work now.
</div>
```

`aria-atomic` controls what gets announced:

`aria-atomic="true"` — When anything inside the region changes, the screen reader re-reads the entire region. Good for status messages where context matters:

HTML Copy

```
  <span id="count">3</span> items in cart
</div>
<!— When count changes from 3 to 4, screen reader says "4 items in cart"
```

`aria-atomic="false"` (default) — Only the changed content is announced. Good for chat messages, activity feeds, and any list where you only want the new item announced:

HTML Copy

```
<!— not atomic: only reads the new message, not the entire history —>
<div aria-live="polite" aria-atomic="false">
  <p>Alice: Hey!</p>
  <p>Bob: Hi there!</p>
  <!— New message added → screen reader only says "Alice: Are you free?"
  <p>Alice: Are you free?</p>
</div>
```

Common patterns in real applications:

Toast notifications:

JSX Copy

```
function ToastContainer() {
  const [toasts, setToasts] = useState([]);

  return (
    <div aria-live="polite" role="status" className="toast-container">
      {toasts.map(toast => (
        <div key={toast.id} className="toast">
          {toast.message}
        </div>
      ))}
    </div>
```

Loading states:

HTML Copy

```
<div aria-live="polite">
  <span aria-busy="true">Loading products ... </span>
</div>

<!— After loading completes, update the content —>
<div aria-live="polite">
  <span>24 products loaded</span>
</div>
```

Character count:

HTML Copy

```
<textarea id="bio" maxlength="280"></textarea>
<div aria-live="polite" aria-atomic="true">
  <span id="char-count">280</span> characters remaining
</div>
```

Critical mistake to avoid: Don't add `aria-live` to an element and inject it into the DOM at the same time. The element must already exist in the DOM (empty is fine), and then you update its content. If you inject a brand new element with `aria-live`, some screen readers won't announce it because the live region wasn't present when they built their accessibility tree.

JAVASCRIPT Copy

```
// Wrong: Creating and injecting the live region AND its content simultane
const alert = document.createElement('div');
alert.setAttribute('aria-live', 'assertive');
alert.textContent = 'Error!';
document.body.appendChild(alert);
// Some screen readers won't announce this
```

```
// Assuming error region already in DOM, then content is updated
const existingAlert = document.getElementById('error-region');
// <div id="error-region" aria-live="assertive"></div> - already in the HT
existingAlert.textContent = 'Error!';
// Screen readers announce "Error!"
```

Follow-up: "What's the difference between `role="alert"` and `aria-live="assertive"`?"

`role="alert"` implicitly sets `aria-live="assertive"` and `aria-atomic="true"`. It also adds an implicit "alert" announcement — screen readers say "Alert: [your message]" instead of just the message. Use `role="alert"` for error messages and critical warnings where the "alert" prefix adds value. Use `aria-live="assertive"` without the role when you want urgency without the "alert" prefix.

Q 2.9 How would you make a custom dropdown/select accessible?

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Atlassian, Swiggy, Microsoft

This is the single most frequently asked accessibility question in frontend interviews. Custom dropdowns are in every UI library, every design system, and every production application — and the vast majority of them are inaccessible. Interviewers use this question to test whether you understand ARIA roles, keyboard interaction, state management, and focus patterns all at once.

The native `<select>` is fully accessible out of the box — keyboard navigable, screen reader compatible, works with form submission. But designers need custom styling, search functionality, multi-select, grouped options, or rich content (icons, descriptions) that native selects can't provide. When you go custom, you need to rebuild all the behavior the browser gives for free.

The markup structure:

HTML Copy

```
  <button
    role="combobox"
    id="country-trigger"
    aria-expanded="false"
    aria-haspopup="listbox"
    aria-labelledby="label-country"
    aria-controls="country-listbox"
    aria-activedescendant=""
  >
    Select a country
  </button>

  <ul
    role="listbox"
    id="country-listbox"
    aria-labelledby="label-country"
    hidden
  >
    <li role="option" id="opt-in" aria-selected="false">India</li>
    <li role="option" id="opt-us" aria-selected="false">United States</li>
    <li role="option" id="opt-uk" aria-selected="false">United Kingdom</li
    <li role="option" id="opt-de" aria-selected="false">Germany</li>
    <li role="option" id="opt-jp" aria-selected="false">Japan</li>
  </ul>
</div>
```

Keyboard interactions you must implement:

JAVASCRIPT Copy

```
const trigger = document.getElementById('country-trigger');
const listbox = document.getElementById('country-listbox');
const options = Array.from(listbox.querySelectorAll('[role="option"]'));
let highlightedIndex = -1;
let isOpen = false;

trigger.addEventListener('keydown', (e) => {
  switch (e.key) {
```

```
    case 'Enter':
      e.preventDefault();
      if (isOpen) {
        selectHighlighted();
        closeDropdown();
      } else {
        openDropdown();
      }
      break;

    case 'ArrowDown':
      e.preventDefault();
      if (!isOpen) openDropdown();
      highlightNext();
      break;

    case 'ArrowUp':
      e.preventDefault();
      if (!isOpen) openDropdown();
      highlightPrevious();
      break;

    case 'Escape':
      if (isOpen) {
        closeDropdown();
        trigger.focus();
      }
      break;

    case 'Home':
      e.preventDefault();
      highlightIndex(0);
      break;

    case 'End':
      e.preventDefault();
      highlightIndex(options.length - 1);
      break;

    default:
      // Type-ahead: pressing "g" jumps to "Germany"
      handleTypeAhead(e.key);
      break;
```

Managing the highlighted state visually AND for screen readers:

JAVASCRIPT Copy

```
function highlightOption(index) {
  // Remove visual highlight from all options
  options.forEach(opt => opt.classList.remove('highlighted'));

  // Add visual highlight to the new option
  options[index].classList.add('highlighted');

  // Scroll the option into view if needed
  options[index].scrollIntoView({ block: 'nearest' });

  // Tell the screen reader which option is currently active
  // The screen reader follows aria-activedescendant, not DOM focus
  trigger.setAttribute('aria-activedescendant', options[index].id);

  highlightedIndex = index;
}
```

The key concept here is `aria-activedescendant`. DOM focus stays on the trigger button. The user arrows through options, but focus never moves. Instead, `aria-activedescendant` points to the currently highlighted option, and the screen reader announces it as if it were focused. This is the standard pattern for combobox widgets.

Opening and closing:

JAVASCRIPT Copy

```
function openDropdown() {
  isOpen = true;
  listbox.hidden = false;
  trigger.setAttribute('aria-expanded', 'true');

  // Highlight the currently selected option, or the first one
  const selectedIndex = options.findIndex(
    opt => opt.getAttribute('aria-selected') === 'true'
```

```
function closeDropdown() {
  isOpen = false;
  listbox.hidden = true;
  trigger.setAttribute('aria-expanded', 'false');
  trigger.setAttribute('aria-activedescendant', '');
  highlightedIndex = -1;
}
```

Selecting an option:

JAVASCRIPT Copy

```
function selectOption(index) {
  // Deselect all
  options.forEach(opt => opt.setAttribute('aria-selected', 'false'));

  // Select the chosen option
  options[index].setAttribute('aria-selected', 'true');

  // Update the trigger button text
  trigger.textContent = options[index].textContent;

  closeDropdown();
  trigger.focus();
}
```

Click outside to close:

JAVASCRIPT Copy

```
document.addEventListener('click', (e) => {
  if (isOpen && !e.target.closest('.custom-select')) {
    closeDropdown();
  }
});
```

Type-ahead — jumping to options by typing:

```
let typeAheadBuffer = '';
let typeAheadTimeout;

function handleTypeAhead(key) {
  if (key.length !== 1) return;  // Ignore non-character keys

  clearTimeout(typeAheadTimeout);
  typeAheadBuffer += key.toLowerCase();

  const match = options.findIndex(opt =>
    opt.textContent.toLowerCase().startsWith(typeAheadBuffer)
  );

  if (match >= 0) {
    if (!isOpen) openDropdown();
    highlightOption(match);
  }

  typeAheadTimeout = setTimeout(() => {
    typeAheadBuffer = '';
  }, 500);
}
```

Why interviewers love this question:

Building an accessible custom dropdown tests everything at once. You need to know ARIA roles (`combobox`, `listbox`, `option`), ARIA states (`aria-expanded`, `aria-selected`, `aria-activedescendant`), keyboard navigation (arrow keys, Enter, Escape, Home, End, type-ahead), focus management (keeping focus on the trigger, using `aria-activedescendant` for virtual focus), and state management (open/closed, highlighted index, selected value). If you can implement this correctly under interview conditions, you understand accessibility deeply enough to handle any widget.

Follow-up: "How would this change for a multi-select dropdown?"

Two differences. First, `aria-multiselectable="true"` on the listbox. Second, pressing Enter or Space toggles the option's `aria-selected` state without closing the dropdown — the user keeps selecting until they press Escape or click outside. The trigger text shows a count ("3 selected") or a comma-separated list of selected values.

Q 2.10 How do you handle accessible form validation?

Difficulty: Senior **Asked at:** Google, Flipkart, Amazon, Razorpay, Microsoft

Most form validation is designed for sighted users — red borders, error text below the field, a warning icon. None of these visual cues communicate anything to a screen reader unless you wire up the correct ARIA attributes. An accessible form needs to answer three questions for screen reader users: "Is this field required?", "Is my input valid?", and "What went wrong?"

A properly accessible form field:

HTML Copy

```html
<div class="form-field">
  <label for="email">
    Email address
    <span aria-hidden="true" class="required-indicator">*</span>
  </label>
  <input
    type="email"
    id="email"
    name="email"
    required
    aria-required="true"
    aria-invalid="false"
    aria-describedby="email-hint email-error"
  />
  <span id="email-hint" class="hint">
    We'll send your order confirmation here
  </span>
  <span id="email-error" class="error" role="alert" aria-live="assertive">
    <!-- Error message will be inserted here by JavaScript -->
  </span>
</div>
```

When the user submits and validation fails:

JAVASCRIPT Copy

```
  const errorElement = document.getElementById(`${field}-error`);

  // Mark the input as invalid — screen reader announces "invalid entry"
  input.setAttribute('aria-invalid', 'true');

  // Insert the error message — role="alert" + aria-live="assertive"
  // means the screen reader announces it immediately
  errorElement.textContent = message;
}

function clearError(field) {
  const input = document.getElementById(field);
  const errorElement = document.getElementById(`${field}-error`);

  input.setAttribute('aria-invalid', 'false');
  errorElement.textContent = '';
}

// Validate on blur (when user leaves the field)
document.getElementById('email').addEventListener('blur', (e) => {
  const value = e.target.value;

  if (!value) {
    showError('email', 'Email address is required');
  } else if (!value.includes('@')) {
    showError('email', 'Please enter a valid email address');
  } else {
    clearError('email');
  }
});
```

How this works for screen readers:

When the user tabs to the email field, the screen reader announces: *"Email address, required, edit text, We'll send your order confirmation here."*

The label and `aria-required` tell them what the field is. The `aria-describedby` links the hint text to the input, so it's read as part of the field description.

announcement. Second, the error message inside the `role="alert"` container is announced immediately (because of `aria-live="assertive"`), even if the user isn't focused on the field.

Error summary pattern — the gold standard for form submission:

When a form has multiple errors, show a summary at the top of the form and move focus to it:

HTML Copy

```
<div
  id="error-summary"
  role="alert"
  aria-live="assertive"
  tabindex="-1"
  hidden
>
  <h2>There are 2 errors in this form</h2>
  <ul>
    <li><a href="#email">Email address is required</a></li>
    <li><a href="#password">Password must be at least 8 characters</a></li
  </ul>
</div>

<form>
  <!— form fields —>
</form>
```

JAVASCRIPT Copy

```
function handleSubmit(e) {
  e.preventDefault();
  const errors = validateForm();

  if (errors.length > 0) {
    const summary = document.getElementById('error-summary');
```

```
    const list = summary.querySelector('ul');
    list.innerHTML = errors
      .map(err => `<li><a href="#${err.field}">${err.message}</a></li>`)
      .join('');

    // Show and focus the summary
    summary.hidden = false;
    summary.focus();  // tabindex="-1" makes this focusable

    // Also mark individual fields as invalid
    errors.forEach(err => showError(err.field, err.message));
  }
}
```

This pattern is used by the UK Government Design System (GOV.UK) and is considered the gold standard for form validation accessibility. The summary gives users a complete overview of all errors, and the links inside jump directly to the problematic fields.

Inline validation timing matters:

Don't validate on every keystroke — that creates a stream of error announcements while the user is still typing. Common approaches:

- **Validate on blur** (when the user leaves the field). This is the most common pattern — the user finishes entering data, and you check it.
- **Validate on submit** with an error summary. Good for simple forms where instant feedback isn't needed.
- **Validate on blur + re-validate on input after an error was shown.** Show the error when the user leaves the field. Once an error is displayed, validate on each keystroke so the error disappears as soon as the input is valid. This gives a responsive "fix confirmed" experience.

JAVASCRIPT Copy

```
input.addEventListener('blur', () => {
  validate();
  hasBeenValidated = true;
});

input.addEventListener('input', () => {
  // Only re-validate on keystroke if we've already shown an error
  if (hasBeenValidated) {
    validate();
  }
});
```

Common mistakes:

Using `placeholder` as the only label. Placeholders disappear when the user starts typing, leaving no label. Screen readers may or may not announce placeholder text. Always use `<label>`.

Using color alone to indicate errors. Red text is invisible to color-blind users. Always include text descriptions alongside visual indicators.

Announcing errors while the user is still typing. Wait until they finish (on blur or on submit), then announce.

Follow-up: "How do you handle validation in React with accessibility?"

Same principles. Use `aria-invalid`, `aria-describedby`, and live regions. React Hook Form and Formik both support adding ARIA attributes to fields. The key addition is using `useRef` to focus the error summary after submission fails:

JSX Copy

```
const summaryRef = useRef(null);

function onSubmit(data) {
  const errors = validate(data);
  if (errors.length > 0) {
    setFormErrors(errors);
    summaryRef.current?.focus();
```

QUICK CHECK — no marks, just for you.

1. A product listing page shows a result count that updates as the user changes filters (for example, Showing 42 results). The update is informational, not urgent. Which aria-live value should the count region use, and why?

- aria-live="assertive", because polite only announces when the element already has DOM focus
- aria-live="off", because polite and assertive both require role="alert" to work
- aria-live="polite", because the screen reader should finish what it is currently saying before announcing the new count
- aria-live="assertive", because the count is new information the user has not heard yet

Check answer

2. A developer creates a fresh div, sets aria-live="assertive" on it, sets its textContent to the error message, and appends it to document.body. Testing shows some screen readers never announce the error. What is the underlying problem?

- assertive live regions only work on elements that are focusable, so the div needs a tabindex
- textContent bypasses the accessibility tree; the region must be populated with innerHTML instead
- A live region only announces when the change happens inside a form element, so appending to document.body is ignored
- The live region must already exist in the DOM (it can be empty) before its content is updated, otherwise some screen readers never registered it as a live region

3. In the accessible custom dropdown pattern from the lesson, the user presses ArrowDown to move through the options in the listbox. Where does DOM focus live while they do this, and how does the screen reader know which option is highlighted?

- ○ Focus stays on the trigger button; aria-activedescendant points at the highlighted option's id and the screen reader announces it as if it were focused
- ○ Focus moves to the listbox container; aria-selected on the highlighted option tells the screen reader what is active
- ○ Focus stays on the trigger button; the screen reader reads the option because scrollIntoView brings it into the viewport
- ○ Focus moves to each option in turn via element.focus(); aria-activedescendant only mirrors that focus for older screen readers

Check answer

‹ Previous
Focus management

Next ›
Layout and visual

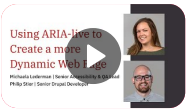
LEARN MORE

ARTICLES & DOCS

ARIA live regions (MDN)
MDN

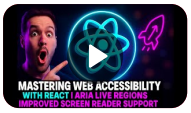
VIDEOS

Frontend System Desi...



aria-live for dynamic pages

YouTube



ARIA live regions in React

YouTube

Hand-picked references for this lesson.